

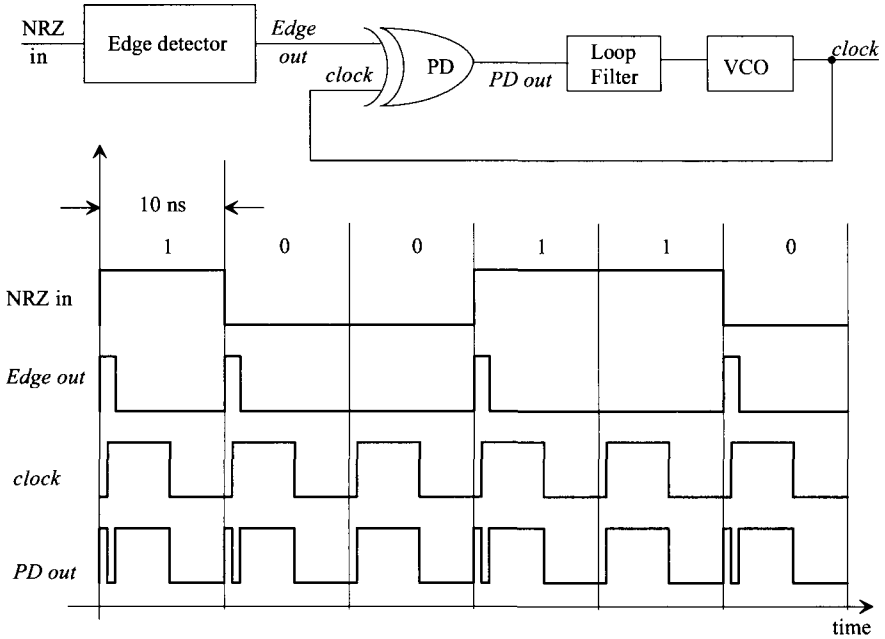


Figure 19.48 Clock-recovery circuit for NRZ using an edge detector. Note that the DPLL is in lock, when the rising edge of clock is centered on the edge output pulse.

Not having the *clock* aligned to the center of the data bit can cause problems in high-speed, clock-recovery circuits. Simply adding a delay in series with the *clock* signal does not solve this problem since the temperature dependence and process variations of the associated circuit do not guarantee proper alignment. What is needed is a circuit that is *self-correcting*, causing the clock signal to align to the center of the data bit independent of the data-rate, the temperature, or process variations.

The Hogge Phase Detector

The PD portion of a self-correcting, clock-recovery circuit is shown in Fig. 19.49 along with associated waveforms for a locked DPLL. Nodes A and B are simply the input NRZ data shifted in time by one-half bit-interval and one bit-interval, respectively. The outputs of the phase detector are labeled *Increase* and *Decrease*. If *Increase* is low more often than *Decrease*, the average voltage out of the loop filter and thus the frequency out of the VCO will decrease. A loop filter that can be used in a self-correcting DPLL is shown in Fig. 19.50a. This filter is the active-PI loop filter discussed in Sec. 19.3.1, with an added input to accommodate both outputs of the PD. Figure 19.50b shows the resulting waveforms in a DPLL where the clock is leading the center of the NRZ data, and thus *Increase* is high less often than *Decrease*. If the NRZ data was lagging the center of the data bit, *Decrease* would be high less often than *Increase*, resulting in an increase in the loop-filter output voltage. Note that in this discussion we have neglected the propagation delays present in the circuit. For a high-speed, self-correcting PD design, we would have to analyze each delay in the PD to determine their effect on the performance of the DPLL.

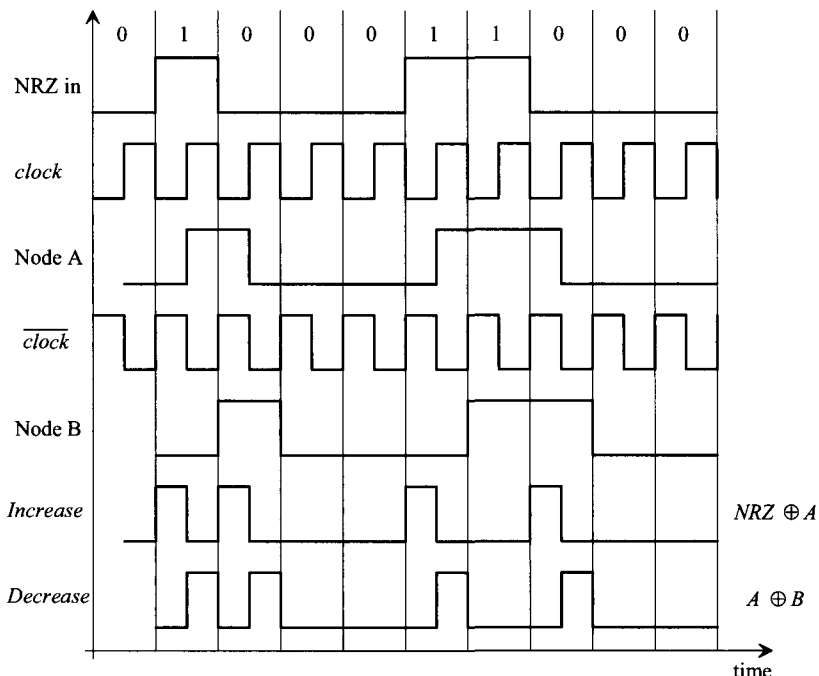
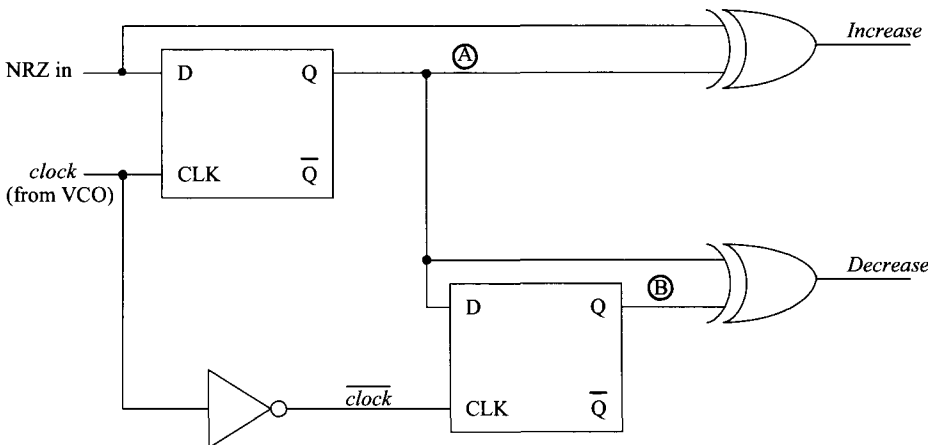
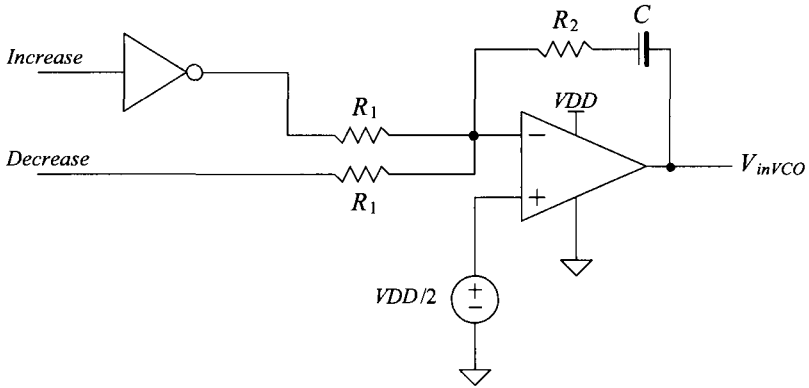
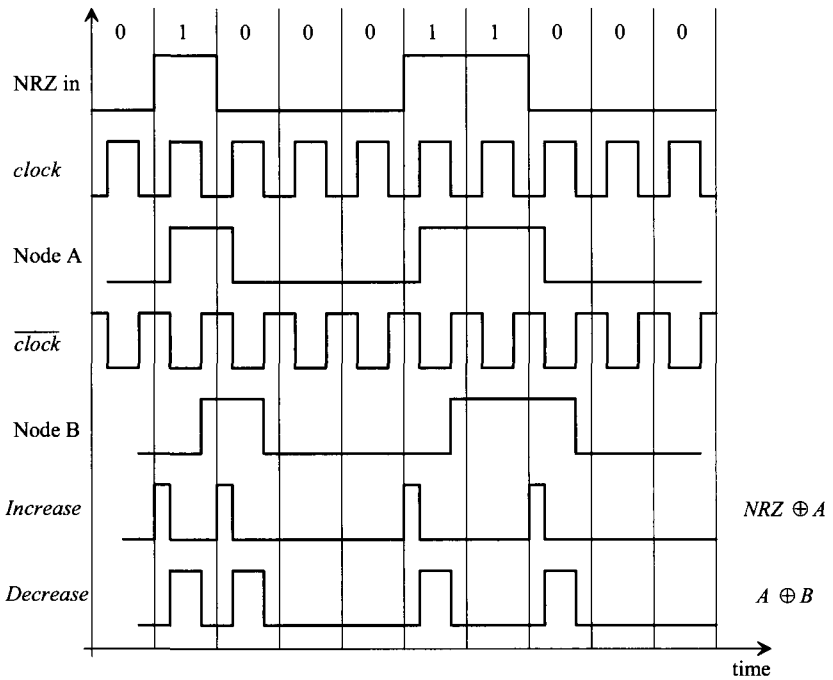


Figure 19.49 The PD (Hogge) portion of a self-correcting, clock-recovery circuit in lock.



(a)



(b)

Figure 19.50 (a) Possible loop filter used in a self-correcting (Hogge) DPLL and (b) waveforms when the loop is not in lock and the clock leads the center of the data.

Jitter

Jitter, in the most general sense, for clock-recovery and synchronization circuits, can be defined as the amount of time the regenerated clock varies once the loop is locked. Figure 19.51a shows the idealized case when the *clock* doesn't jitter, while Fig. 19.51b shows the actual situation where the clock-rising edge moves in time (jitters). In these figures, the oscilloscope is triggered by the rising edge of the *data*. In the following discussion, we neglect power supply and oscillator noise, that is, we assume that the oscillator frequency is an exact number that is directly related to the VCO input voltage. In the section following this one, we cover delay-locked loops and further discuss the limitations of the VCO.

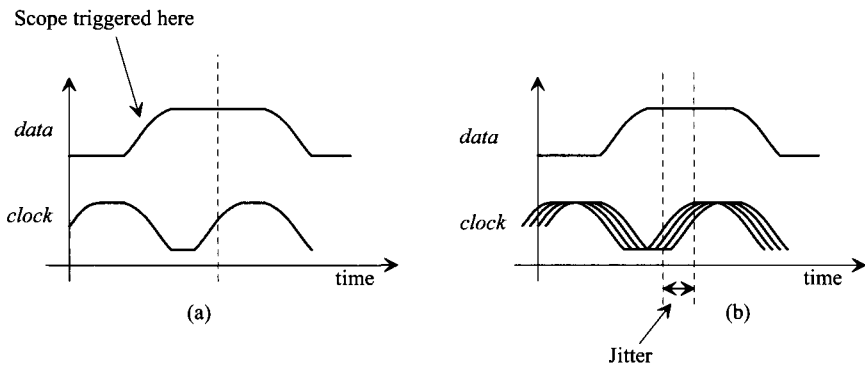


Figure 19.51 (a) Idealized view of clock and data without jitter and (b) with jitter.

Consider using the charge pump with the self-correcting PD shown in Fig. 19.52. When the loop is locked (from Fig. 19.49), both increase and decrease occur (with the same width) for every transition in the incoming data. Note that unlike a PFD/charge pump combination where the output of the charge pump remains unchanged when the loop is locked, the self-correcting PD/charge-pump combination generates a voltage ripple on the input of the VCO (similar to the XOR PD with RC or Active PI filter)⁴. Let's assume that this ripple is 10 mV and use the values for VCO gain and frequencies given in Ex. 19.5 to illustrate the resulting jitter introduced into the output clock. The change in output frequency resulting from this ripple is $10\text{mV} \cdot (1.57 \times 10^9 \text{ radians/V} \cdot \text{s}) \cdot (1/2\pi)$ or 2.5 MHz. This means that the output of the DPLL will vary from, say, 100 MHz to 102.5 MHz. In terms of a jitter specification (see Eq. (19.41)), the clock jitter is (roughly) 250 ps (2.5% of the output clock's period).

From this example, *data dependent jitter*, can be reduced by

1. *Reducing the gain of the VCO.* Ripple on the input of the VCO has less of an effect on the output frequency. The main disadvantage, in the most general sense, of reducing the gain of the VCO is that the range of frequencies the DPLL can

⁴ Of course, the self-correcting PD should not be used in most frequency synthesis applications. Similarly, the PFD should not be used in most clock-recovery applications.

lock up on is reduced. Also, the VCO gain strongly affects the ability to fabricate the VCO without postproduction tuning.

2. *Reducing the bandwidth of the loop filter.* This has the effect of reducing the actual ripple on the input of the VCO. The main drawback here is the increase in chip real estate needed to realize the larger components used in the filter.
3. *Reducing the gain of the PD.* This also has the effect of reducing the ripple on the input of the VCO. This method is, in general, the easiest when using the charge pump since reducing the gain is a simple matter of reducing I_{pump} . Substrate noise can become more of a factor in the design.

Another way of stating the above methods of reducing jitter is simply to say that the forward loop gain of the DPLL, that is, $K_{PD}K_FK_{VCO}$, should be made small. The main problems with using small forward loop gain are the reductions in lock range and pull-in range coupled with the associated increases in pull-in and lock times.

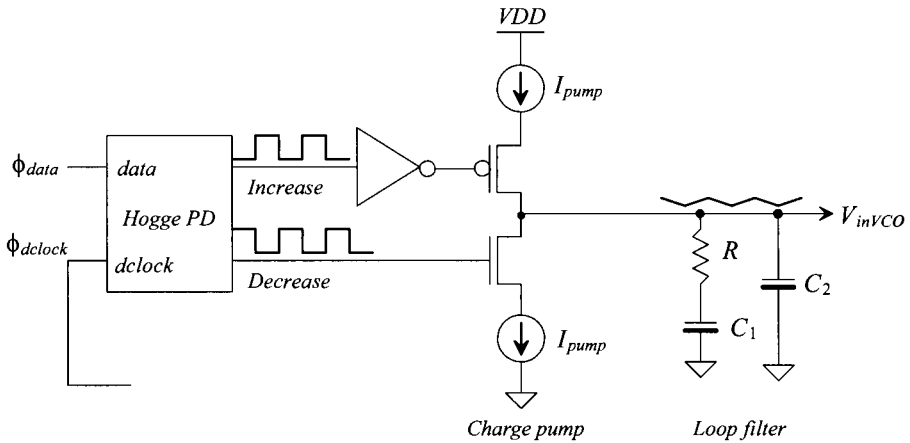


Figure 19.52 Self-correcting PD with charge pump output.

19.5 Delay-Locked Loops

Problems with PLL output jitter resulting from the VCO output frequency changing (often called oscillator or phase noise) with a constant input voltage ($V_{inVCO} = \text{constant}$) has led to the concept of a delay-locked loop (DLL). Figure 19.53 shows the basic block diagram of a DLL. Assuming that a reference clock is available at exactly the correct frequency, the input data is delayed through a voltage-controlled delay line (VCDL) a time t_o until it is synchronized with the reference clock. Jitter is reduced by using an element, the VCDL, that does not generate a signal (like the VCO did). The transfer function ϕ_{clock}/ϕ_{out} is zero (the phase of the reference clock is taken as the reference for the other signals in the DLL, i.e., $\phi_{clock} = 0$), so that oscillator noise and the resulting jitter are not factors in DLL design. The jitter considerations discussed in the last section, however, are still a concern since any ripple on the output of the loop filter will cause jitter.

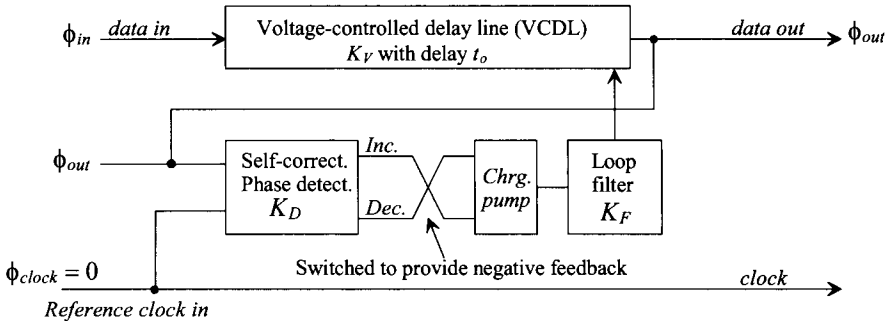


Figure 19.53 Block diagram of a delay-locked loop.

The phase (in radians) of the input data is related to the phase of the output data by

$$\phi_{out} = \phi_{in} + t_o \cdot \frac{2\pi}{T_{clock}} \quad (19.63)$$

where T_{clock} is the period of the reference clock (or half of the period of the *data in* for a string of alternating ones and zeros). The gain of the VCDL can be written in terms of the delay, t_o , by

$$t_o = K_V \cdot V_{indel} \quad (19.64)$$

where K_V has units of seconds/V and V_{indel} is the voltage input to the VCDL from the loop filter. The minimum and maximum delays of the VCDL should, in general, lie between $T_{clock}/2$ and $1.5T_{clock}$ for proper operation. The output of the loop filter (input to the VCDL) can be written as

$$V_{indel} = \phi_{out} \cdot K_D \cdot K_F \quad (19.65)$$

The overall transfer function may now be written as

$$\frac{\phi_{out}}{\phi_{in}} = \frac{1}{1 - K_D K_F K_V \cdot \omega_{clk}} \quad (19.66)$$

where $\omega_{clk} = 2\pi/T_{clock}$. The gain of the self-correcting PD with charge-pump output, with the help of Fig. 19.54 and noting that *Increase* and *Decrease* can occur at the same time, is

$$K_D = -\frac{I_{pump}}{\pi} \text{ (amps/radian)} \quad (19.67)$$

The negative sign is the result of switching the *Increase* and *Decrease* outputs of the self-correcting PD when connecting to the charge pump. This switch is required to provide negative feedback around the loop. Another benefit of the DLL is that the loop filter can be a simple capacitor, which results in a first-order feedback loop, that is,

$$K_F = \frac{1}{sC_1} \quad (19.68)$$

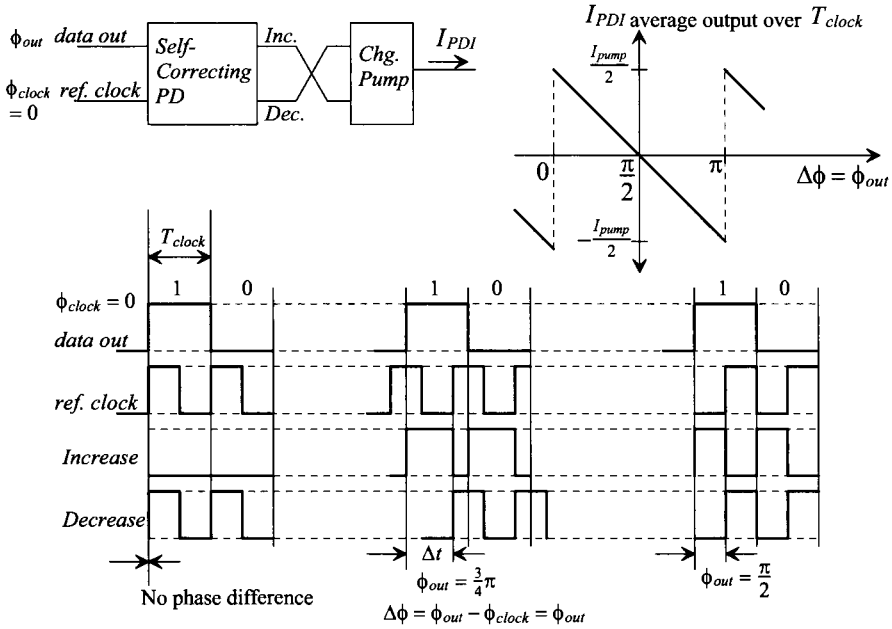


Figure 19.54 Self-correcting PD output for various inputs (assuming input data is a string of alternating ones and zeros).

The transfer function of the DLL relating the input data to the time-shifted output is

$$\frac{\phi_{out}}{\phi_{in}} = \frac{1}{1 + \frac{I_{pump}}{\pi} \cdot \frac{1}{sC_1} \cdot K_V \cdot \omega_{clk}} = \frac{s}{s + K_V \cdot \frac{2I_{pump}}{C_1 T_{clock}}} \quad (19.69)$$

We know that the frequency of the reference clock must be exactly related to the frequency of the input data. However, there will exist instantaneous changes in the phase of the input data which the output of the DLL should follow. Modeling instantaneous changes in ϕ_{in} by $\Delta\phi_{in}/s$ (a step function with an amplitude of $\Delta\phi_{in}$), we get a change in output phase given by

$$\Delta\phi_{out} = \frac{\Delta\phi_{in}}{s + K_V \cdot \frac{2I_{pump}}{C_1 T_{clock}}} \quad (19.70)$$

The time it takes the DLL to respond to an input step in phase is simply

$$T_r = 2.2 \cdot \frac{C_1 T_{clock}}{K_V \cdot 2I_{pump}} = \text{number of clock cycles} \cdot T_{clock} \quad (19.71)$$

This time can be decreased by making C_1/I_{pump} small, which from our discussion in the last section, has the result of increasing the output pulse jitter (i.e., jitter dependent on the input data pattern). Decreasing C_1/I_{pump} , increases the ripple on the control voltage of the VCDL. Similarly, increasing K_V (the time/volt delay of the VCDL) increases jitter since a given ripple on the control voltage of the VCDL will have a larger effect on the delay. Again, trade-offs must be made between responses to input variations and output jitter.

Delay Elements

The VCDL is an important component of the DLL. Figure 19.55a shows the basic implementation of a VCDL using adjustable delay inverters. The last two inverters in the VCDL ensure that clean digital signals are output from the line. Figure 19.55b shows the circuit schematics for possible delay elements. The first delay element should be recognized as a current-starved inverter discussed earlier in the chapter. The second delay element is nothing more than an inverter with a variable load. In practice, these delay elements are rarely used because of the susceptibility to noise and power supply variations. Rather, fully-differential delay elements are used.

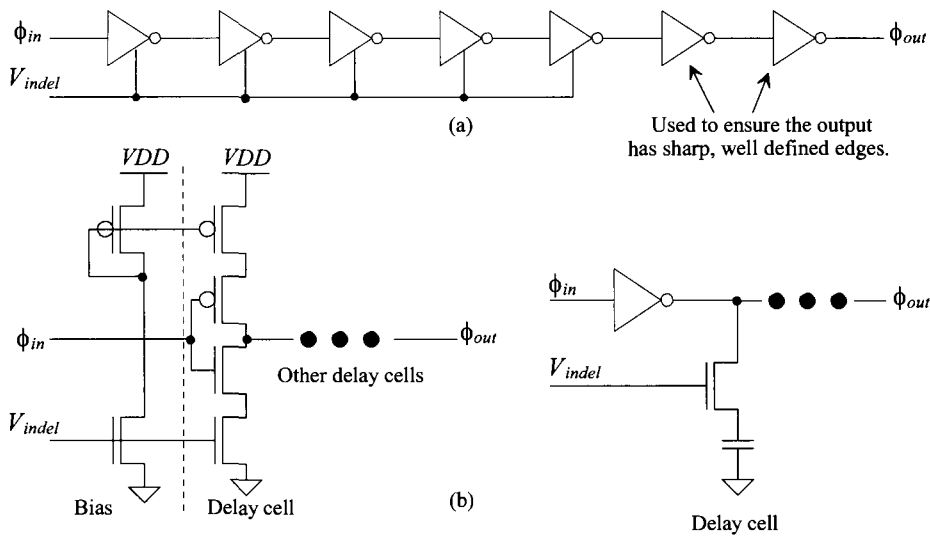


Figure 19.55 (a) VCDL made using inverter delay cells and (b) possible delay cells.

Figure 19.56 shows the connection of a fully differential VCDL. Using any number of stages, a fully differential VCO can be implemented with the delay elements (and the proper feedback), while using an even number with the inverting (noninverting) output fed back and connected to the noninverting (inverting) input in-phase (I) and quadrature (Q) signals can be generated.

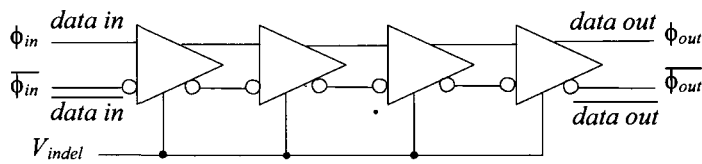


Figure 19.56 Implementation of a fully differential VCDL.

Practical VCO and VCDL Design

An example of a practical delay stage is shown in Fig. 19.57. The control voltage (that controls the delay if used in a VCDL or the oscillation frequency if the element is used in a VCO) can be generated from some linear voltage to current converter (like the one seen in Fig. 19.17 using a resistor, M5R, and M6R). The reason that this circuit is labeled “practical” can be understood by first considering what happens when there is noise on V_{DD} . The noise causes a voltage variation across the PMOS current source, MPC. Ideally, this doesn’t change the current through MPC. The output voltages of the delay cell swing between V_{REF} and ground. If there is noise on ground, it feeds equally into each output. This common-mode noise is then, ideally, rejected by the differential amplification action of the next stage. The bias circuit is a half-replica of the delay stage and is used to bias the delay elements so that the swing is up to V_{REF} when the gate of one of the PMOS switches is at ground.

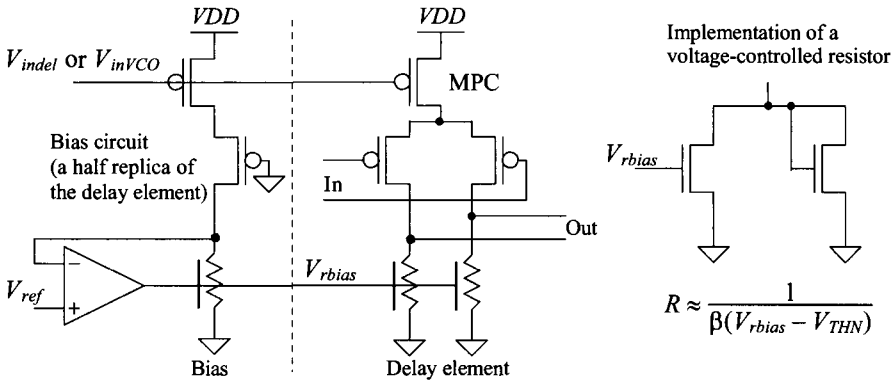


Figure 19.57 A differential delay element based on a voltage-controlled resistor. The bias circuit adjusts the value of the resistors used in the delay elements to sink the current sourced by the p-channel MOSFETs.

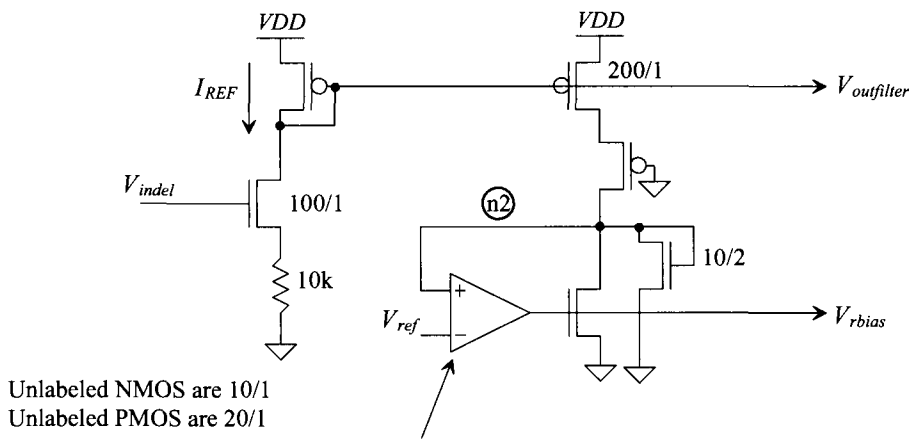
19.6 Some Examples

To finish up this chapter, let’s present some examples with discussions and simulations.

19.6.1 A 2 GHz DLL

To begin, let’s design a 500 ps delay line for a DLL that is used for *clock synchronization*. Let’s design the delay line so that when it is used in a DLL, with the input and output of the delay line synchronized, we have eight phases of the input clock (we need eight stages in the delay line).

The first circuits we need to design are the bias circuit and the voltage-to-current converter used to generate $V_{outfilter}$ and V_{rbias} (see Fig. 19.57). We’ll use the basic schemes of Fig. 19.17 and 19.57, as seen in Fig. 19.58. The important issues concerning this circuit are: 1) the linearity of the V_{indel} voltage against the generated I_{REF} , 2) the sensitivity of I_{REF} to changes in V_{DD} , and 3) how well the amplifier regulates node n2 to V_{REF} .



PMOS self-biased diff-amp of Fig. 18.21 without the inverter on its output.

Figure 19.58 Bias circuit for a 500 ps delay line.

Figure 19.59 shows the simulation results for the bias circuit in Fig. 19.58. In (a) the x-axis is the delay line's control voltage, V_{indel} , while the y-axis is the generated reference current. Also seen in the figure, although hard to discern, is the change in VDD from 900 mV to 1.1 V (VDD is changed from 900 mV to 1.1 V in 50 mV steps, while V_{indel} is swept from 300 mV to 800 mV). Figure 19.59b shows how well node n2 is regulated to V_{REF} . V_{REF} can be generated with the beta-multiplier discussed in the next chapter (see Eq. (20.38) and Fig. 20.22).

Note that we aren't setting a lower value of current in this circuit like we did in Fig. 19.25. If we were to do this (which may be useful to reduce jitter since the resulting delay line or VCO would have lower gain), we would use a topology like the one seen in Fig. 19.60. The reference voltage is used to set the minimum current (when V_{indel} is small) through R_{low} . The problem with setting the lower current with the method seen in Fig. 19.25 is that any changes in VDD feed directly across R_{low} and change the bias current. Using the method in Fig. 19.60 with wide NMOS devices, the lower current is (roughly) $(V_{REF} - V_{THN})/R_{low}$.

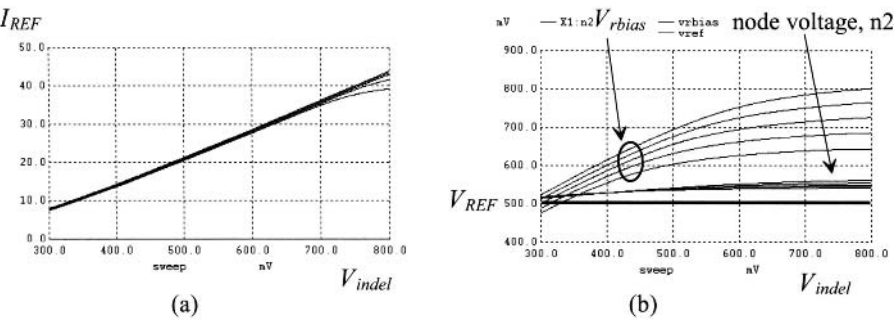


Figure 19.59 The performance of the bias circuit in Fig. 19.58.

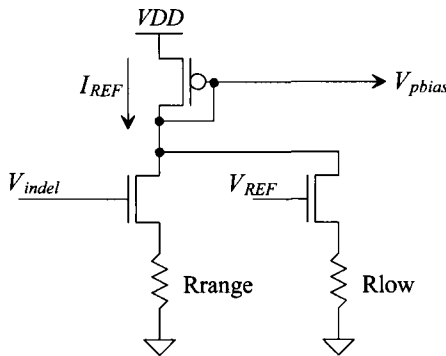


Figure 19.60 Lowering the current range in the bias circuit.

A schematic of the delay line is seen in Fig. 19.61. Notice that the current in the each delay stage was sized up by a factor of ten (the current sources in the delay cell are sized 200/1). This VCDL was simulated using the topology seen in Fig. 19.62. The control voltage was set to $V_{DD}/2$ while the differential inputs were generated using an inverter and a transmission gate (to attempt to equalize the delays that the input signal sees to the VCDL). The outputs of the even stages are shown in this figure. Notice how they are evenly spaced. Notice, also, that the outputs only swing up to V_{REF} ($= 500$ mV here). As discussed earlier, this was done to minimize the effects of power supply and ground noise on the delay of the circuit.

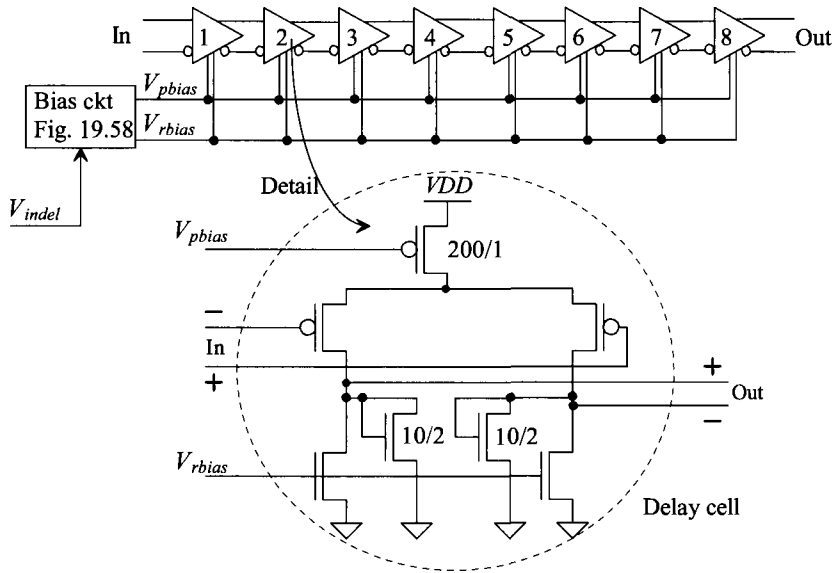


Figure 19.61 An eight-stage VCDL.